

流式语音识别模型性能仿真与并发评测系统 软件设计说明书

编写人：李鲲程

编写时间：2026.1.9

目录

- 1. 软件名称 3
- 2. 软件概述 3
- 3. 软件运行环境 4
- 4. 系统总体设计 4
- 5. 系统功能设计 7
 - 5.1. 评测调度控制模块设计 7
 - 5.1.1. 主程序入口设计 8
 - 5.1.2. 关键类定义：EVALUATIONCONTROLLER 9
 - 5.1.3. 调度层与执行层解耦机制 10
 - 5.2. 流式语音仿真模块设计 11
 - 5.3. 模型与算法执行模块设计 13
 - 5.4. 指标计算与统计模块设计 15
 - 5.5. 结果输出模块设计 16
- 6. 系统关键技术实现 18
 - 6.1. FASTER-WHISPER 准实时分片执行机制 18
 - 6.2. 滑动窗口重叠优化算法 19
 - 6.3. SHERPA-ONNX+BERT 语义增强机制 20
 - 6.4. 并行实时指标采集与统计机制 21
- 7. 系统运行流程 22
- 8. 软件特点与创新点 25
 - 8.1. 离线流式仿真技术 25
 - 8.2. 跨架构统一评测框架 26
 - 8.3. 语义后处理性能损耗量化机制 26
 - 8.4. 高并发负载仿真能力 26
- 9. 结语 26

1. 软件名称

全称： 流式语音识别模型性能仿真与并发评测系统

简称： Streaming-ASR-SimBench

版本号： V1.0

2. 软件概述

本软件是一套面向实时会议语音转写场景的流式语音识别模型性能仿真与并发评测系统。针对流式（Streaming）与非流式（Non-Streaming）语音识别模型在实时处理能力与系统吞吐能力方面的差异，系统构建了离线流式仿真引擎，通过对标准语音数据集中的 WAV 文件进行分帧切片，并以真实语速节流发送，在无需物理麦克风及网络环境的条件下，精确复现真实会议场景中的连续语音流输入特征，实现高保真流式语音输入模拟。

系统兼容 Transducer 与 AED 双架构模型技术路线，集成 Sherpa-ONNX（流式 Transducer 架构）与 Faster-Whisper（非流式 AED 架构）两类主流语音识别模型，并内置基于 BERT 的准实时语义后处理模块，用于评估标点恢复和语义完整性对系统性能的影响。通过统一评测底座与多流轮询调度引擎，系统可自动化执行单流延迟测试与多路高并发压力测试，动态增加并发请求数量，并在 $RTF \approx 1.0$ 条件下判定模型在特定 GPU 环境中的最大实时并发处理能力。

在性能评测方面，系统可实时采集并统计词错误率（WER）、首字上屏延迟（TTFT）以及实时率（RTF）等关键指标，构建覆盖识别精

度、响应延迟和实时处理能力的综合评价体系。评测过程支持在 LibriSpeech、TED-LIUM 及 AMI Corpus 等标准数据集上执行精度验证，并在典型数据集上开展并发能力建模分析。该系统可为工业级低延迟、高并发会议字幕系统的架构设计与部署规模规划提供定量化性能依据与工程决策支持。

3. 软件运行环境

操作系统：Ubuntu 20.04.6 LTS

运行环境：Python 3.12

核心算法库：

(1) 深度学习框架：PyTorch 2.1+ (CUDA 11.8/12.2)

(2) 模型推理：Sherpa-onnx, Faster-Whisper, ONNX Runtime
GPU

(3) NLP 处理：Transformers, DeepMultilingualPunctuation

(4) 信号处理：NumPy, SoundFile, Librosa

并发支持：Python Asyncio

硬件环境：

(1) 计算单元：支持 CUDA 的 NVIDIA 显卡 (显存 $\geq$ 8GB)

(2) 处理器：多核 CPU (建议 16 核以上，用于数据加载与调度)

4. 系统总体设计

本系统是一套面向实时会议语音转写场景的高保真性能仿真与评测软件工具，旨在对流式与非流式语音识别模型在不同负载条件下的

准确性与实时性能进行系统化量化分析。系统围绕会议字幕场景中“低延迟、高准确、可并发”的核心需求，构建了完整的评测调度、流式仿真、模型执行、指标统计与结果输出闭环体系，可在离线环境下模拟真实会议语音流输入过程，对词错误率（WER）、实时率（RTF）以及首字上屏延迟（TTFT）等关键指标进行精确测量。

系统采用模块化分层架构设计，如图 4-1 所示，整体由评测调度控制模块、流式语音仿真模块、模型与算法执行模块、指标计算与统计模块以及结果输出模块五部分组成。评测调度控制模块负责模型选择、数据集选择、指标选择及并发数设置，实现不同评测场景的统一入口控制；流式语音仿真模块通过对 WAV 音频进行分帧切片与时间节流控制，模拟真实语音流输入，并在并发测试场景下构建多路流式实例；模型与算法执行模块统一封装 Faster-Whisper 准实时 1 秒分片模式、Faster-Whisper+滑动窗口降低截断问题的增强模式、Sherpa-ONNX 流式 Transducer 推理流程以及 Sherpa-ONNX+BERT 语义优化作为独立模型执行模式，实现不同技术路线的统一调用。

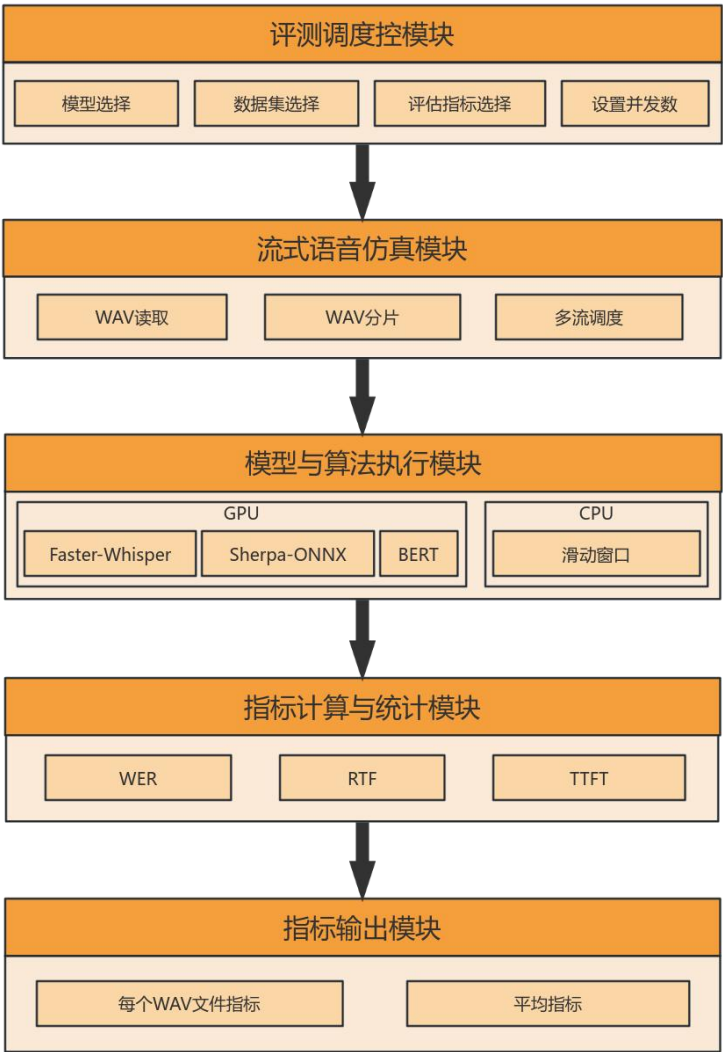


图 4-1 系统架构图

指标计算与统计模块采用并行监测机制，自流式语音仿真阶段开始采集关键时间节点数据，对模型执行过程进行实时监听，从而准确计算 TTFT 与 RTF 等时间类指标，并结合标准文本计算 WER 值。结果输出模块对单文件指标及批量平均指标进行统一整理与输出，实现从评测执行到性能分析的完整闭环。

通过上述结构设计，本系统实现了模型架构解耦、输入机制仿真化、指标统计实时化以及评测流程自动化，能够在多种并发条件下评

估不同语音识别模型的性能表现，为工业级会议字幕系统的构建提供可靠的数据支撑。

5. 系统功能设计

5.1. 评测调度控制模块设计

评测调度控制模块是本系统的统一入口与核心控制中枢，负责对整个评测流程进行参数配置与执行调度。该模块实现模型类型选择、数据集选择、评估指标选择以及并发数设置等功能，通过统一的参数管理机制将不同评测场景抽象为可配置任务实例。

在数据集支持方面，系统内置 LibriSpeech、TED-LIUM 及 AMI Corpus 三类标准公开语音数据集，并提供统一的数据目录解析与音频加载接口。用户可在调度模块中通过配置方式选择不同数据集执行评测任务。其中，词错误率（WER）指标支持在上述三类数据集上进行批量评测；实时率（RTF）与首字上屏延迟（TTFT）指标面向流式性能评估场景，当前基于 LibriSpeech 数据集执行统一基准测试。数据集与指标的适用范围均可通过参数配置进行控制，支持后续扩展。

实际运行流程如图 5-1 所示，调度模块首先读取用户设定的评测参数，包括所选语音识别模型（如非流式 Faster-Whisper 或流式 Sherpa-ONNX）、测试数据集类型、所需计算的评估指标（WER、RTF、TTFT）以及并发任务数量。随后，根据并发数参数创建相应数量的独立流式任务实例，并将统一配置参数分发至流式语音仿真模块与模型执行模块。通过集中控制与统一调度机制，系统能够在单流延迟测试

与多流并发压力测试之间灵活切换，实现不同负载条件下的自动化评测。

该模块在架构层面起到“解耦控制”的作用，不直接参与语音处理与指标计算，而是通过任务编排与参数分发确保各功能模块在统一框架下协同运行，从而提高系统的可扩展性与可维护性。

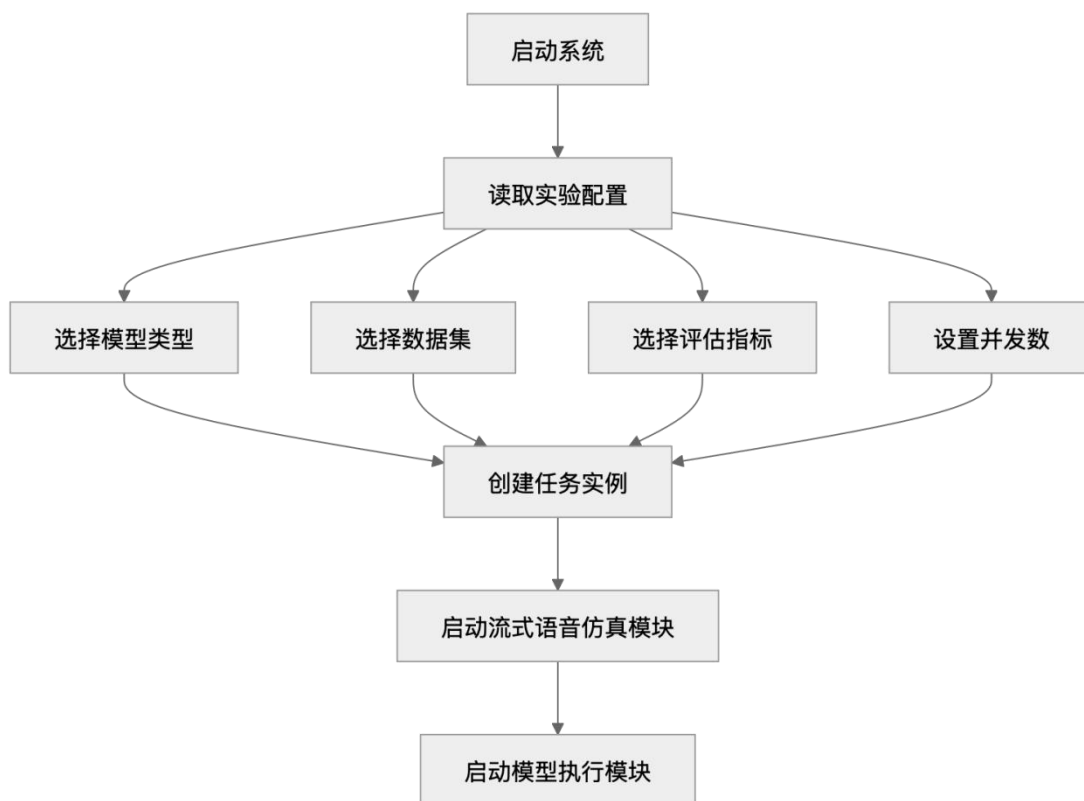


图 5-1 评测调度控制模块运行流程图

5.1.1. 主程序入口设计

系统通过主控制程序 `main_control.py` 作为统一启动入口，采用命令行参数方式实现评测任务配置。用户可通过参数组合定义模型类型、数据集类型、评测指标以及并发数量，实现灵活可配置运行。

```
1 python main_control.py \  
2     --model fw-slide \  
3     --dataset librispeech \  
4     --metrics wer,rtf,ttft \  

```


5 `--concurrency` 5

参数说明

参数	说明
<code>--model</code>	模型类型 (fw / fw-slide / sherpa / sherpa-bert)
<code>--dataset</code>	数据集名称 (librispeech / tedlium / ami)
<code>--metrics</code>	评测指标选择 (wer / rtf / ttft)
<code>--concurrency</code>	并发任务数量

通过 `argparse` 参数解析机制，系统自动生成参数帮助信息 (`--help`)，提升系统可用性与规范性。

5.1.2. 关键类定义：EvaluationController

评测调度控制模块的核心控制逻辑封装于 `EvaluationController` 类中。该类负责：

- a) 参数解析与合法性校验
- b) 数据集加载
- c) 模型实例构建
- d) 并发任务创建
- e) 调用指标统计模块
- f) 汇总并输出最终结果

核心伪代码如下：

```
1 class EvaluationController:
2
3     function __init__(config):
4         self.model_type = config.model
5         self.dataset_type = config.dataset
6         self.metrics = config.metrics
7         self.concurrency = config.concurrency
8
9         self.dataset = DatasetLoader(self.dataset_type)
10        self.model = ModelFactory.create(self.model_type)
11        self.metrics_manager = MetricsManager(self.metrics)
```

```
12
13     function run():
14         pool = ThreadPool(self.concurrency)
15
16         for sample in self.dataset:
17             pool.submit(self.evaluate_task, sample)
18
19         pool.wait_all()
20
21         summary = self.metrics_manager.aggregate()
22         ResultExporter.export(summary)
23
24     function evaluate_task(sample):
25         simulator = StreamingSimulator(sample)
26
27         self.metrics_manager.start(sample.id)
28
29         for chunk in simulator.stream():
30             result = self.model.process(chunk)
31
32             if result.first_token:
33                 self.metrics_manager.record_ttft(sample.id)
34
35         self.metrics_manager.finish(sample.id, sample.duration)
```

5.1.3. 调度层与执行层解耦机制

EvaluationController 不直接参与语音识别算法执行,而是通过统一接口调用模型执行模块。模型具体实现由 ModelFactory 构造,不同模型之间采用统一 process() 接口对外暴露服务。这种设计实现了:

- a) 控制层与算法层分离
- b) 评测逻辑与识别逻辑解耦
- c) 支持新模型平滑扩展
- d) 支持新指标扩展

伪代码如下:

```
1 class ModelFactory:
2
3     function create(model_type):
4
5         if model_type == "fw":
6             return FasterWhisperModel()
7
8         if model_type == "fw-slide":
9             return SlidingWindowWrapper(FasterWhisperModel())
10
11        if model_type == "sherpa":
12            return SherpaStreamingModel()
13
14        if model_type == "sherpa-bert":
15            base = SherpaStreamingModel()
16            return BertEnhancerWrapper(base)
17
18 class BaseModel:
19
20     function process(audio_chunk):
21         return recognition_result
```

5.2. 流式语音仿真模块设计

流式语音仿真模块用于在离线环境下模拟真实会议语音流输入过程，是系统实现高保真实时测试的关键组成部分。该模块通过读取标准 WAV 音频文件，对音频数据进行固定帧长分片，并按照设定的时间节流策略逐帧输出，从而模拟麦克风或网络传输中的连续语音输入行为。不同模型模式对应不同分片粒度，其中 Sherpa-ONNX 模式采用 0.1 秒级分片输入，而 Faster-Whisper 模式采用 1 秒级分片输入。

关键类定义：StreamingSimulator:

```
1 class StreamingSimulator:
2
3     function __init__(audio_sample, chunk_size):
4         self.audio = audio_sample
5         self.chunk_size = chunk_size
```

```
6         self.pointer = 0
7
8     function stream():
9         while pointer < audio.length:
10             chunk = audio.slice(pointer, chunk_size)
11             pointer += chunk_size
12             yield chunk
```

在单流测试场景中，模块按时间顺序逐帧送入模型；在多流并发场景下，根据调度模块传入的并发数参数创建多个独立音频流实例，并通过协程或轮询机制实现多路流式数据的并行推进，运行流程如图 5-2 所示。

在时间维度上，流式语音仿真模块不仅负责数据输入，还作为实时性指标采集的起始点。当首帧语音数据生成时，系统同步启动时间采集机制，用于后续 TTFT 与 RTF 的统计计算。通过严格控制帧间时间间隔，系统能够在无需真实硬件设备的情况下复现会议语音输入节奏，从而提高评测的可重复性与可控性。

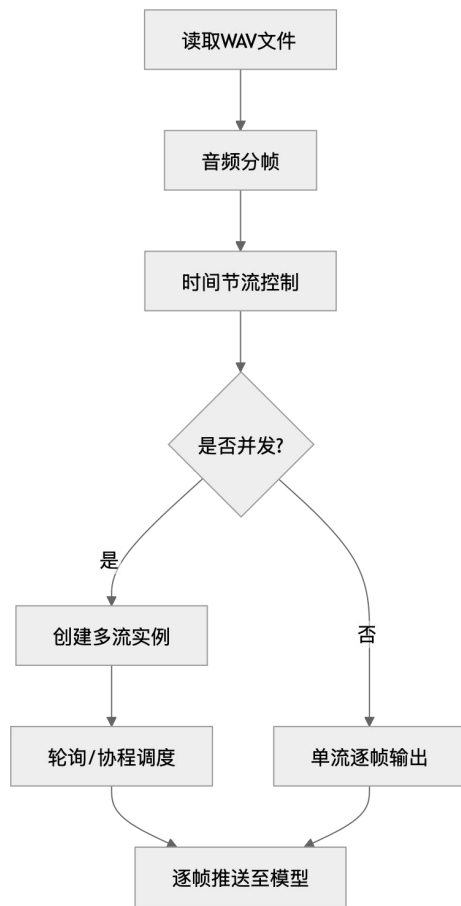


图 5-2 流式语音仿真模块运行流程图

5.3. 模型与算法执行模块设计

模型与算法执行模块是系统的核心计算单元，负责对流式语音仿真模块产生的音频分片数据进行识别推理。本系统在架构层面将不同技术路线与优化策略抽象为四种并列的模型执行模式，分别为：Faster-Whisper 准实时分片模式、Faster-Whisper 滑动窗口优化模式、Sherpa-ONNX 流式模式以及 Sherpa-ONNX+BERT 语义增强模式。四种模式在系统中作为独立选项存在，由评测调度控制模块统一选择并加载：

a) 在 Faster-Whisper 准实时分片模式下，系统将音频数据按 1 秒为单位进行分片输入，每个分片独立调用模型进行推理。通过连续分片调用机制，使原本离线推理结构的模型能够在实时语音转写场景中实现阶段性准实时输出效果。

b) 在 Faster-Whisper 滑动窗口优化模式下，系统同样采用 1 秒分片输入机制，但在推理过程中引入固定长度的历史重叠区，构建滑动窗口结构进行重复覆盖计算。该策略能够缓解语句边界截断问题，提高长语句识别的连续性与稳定性，同时保持准实时输出节奏。

c) 在 Sherpa-ONNX 流式模式下，系统按照 0.1 秒为单位逐帧输入音频数据，利用 Transducer 架构的流式解码特性实现增量推理与连续输出，能够真实反映流式模型在会议实时字幕场景中的响应能力。

d) 在 Sherpa-ONNX+BERT 模式下，系统首先采用 0.1 秒分片方式进行流式解码，在获得阶段性识别文本后，引入 BERT 语义模型对文本进行准实时语义优化与结构修正。该模式作为独立算法选项存在，用于评估语义增强策略对识别准确率与实时性能的综合影响。

模型与算法执行模块运行流程如图 5-3 所示，四种模式均通过统一接口封装，接收来自流式语音仿真模块的音频分片数据，并向指标计算与统计模块输出识别结果及时间节点信息。指标模块在模型执行过程中并行监听关键事件，从而实现 TTFT 与 RTF 等实时指标的精确采集。通过这种并列抽象设计，系统实现了不同分片粒度与不同算法策略的统一调度与横向对比。

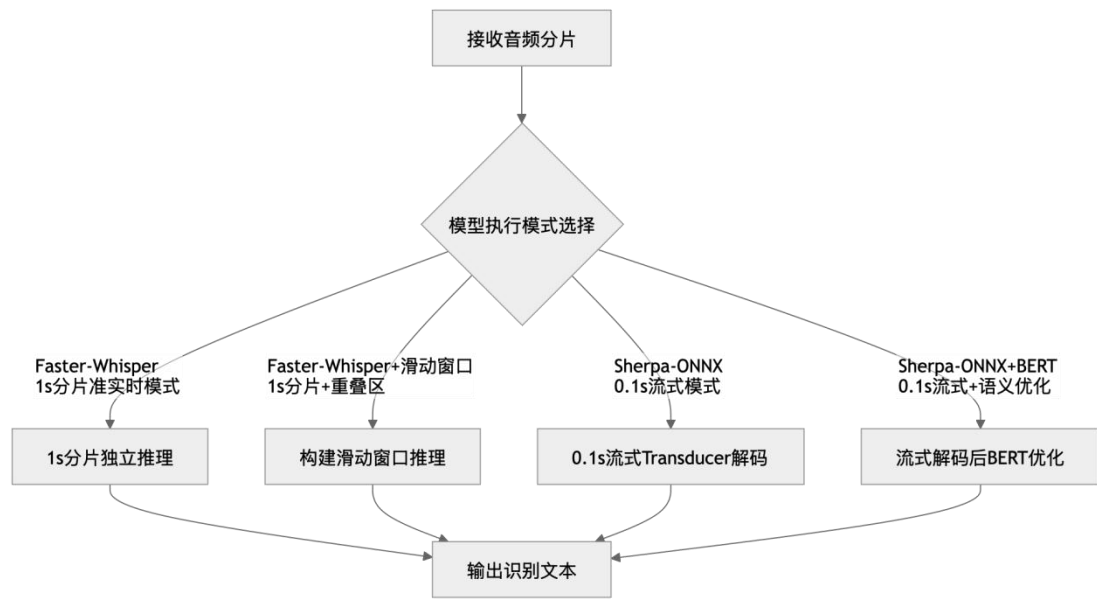


图 5-3 模型与算法执行模块运行流程图

5. 4. 指标计算与统计模块设计

指标计算与统计模块负责对系统运行过程中的关键性能数据进行实时采集与计算。该模块采用并行监听机制，与流式语音仿真模块和模型执行模块同步运行。由于不同模型执行模式采用不同分片粒度，TTFT 统计起点统一定义为首个音频分片生成时刻。系统在首帧语音生成时启动时间计时器，并在首个识别文本输出时记录时间差，用于计算首字上屏延迟（TTFT）。同时，系统在推理结束后根据模型总耗时与音频总时长计算实时率（RTF）。对于准确率评估，模块将模型输出文本与标准文本进行对齐比对，计算插入、删除与替换错误数量，从而得出词错误率（WER）。

关键类定义：MetricsManager：

```
1 class MetricsManager:
2
3     function start(sample_id):
4         start_time[sample_id] = now()
5
```

```

6   function record_first_token(sample_id):
7       if not recorded:
8           ttft[sample_id] = now() - start_time[sample_id]
9
10  function finish(sample):
11      total_time = now() - start_time[sample.id]
12      rtf[sample.id] = total_time / sample.duration
13
14  function compute_wer(reference, hypothesis):
15      return edit_distance(reference, hypothesis) / len(reference)
16
17  function aggregate_results():
18      output average(WER), average(RTF), average(TTFT)

```

该模块并非事后统计，而是在系统执行全过程中持续监听关键时间节点与输出结果，实现实时数据采集与统计计算，运行流程如图 5-4 所示。通过统一的指标计算框架，系统能够保证不同模型与不同测试场景下的评测结果具有一致性与可比性。

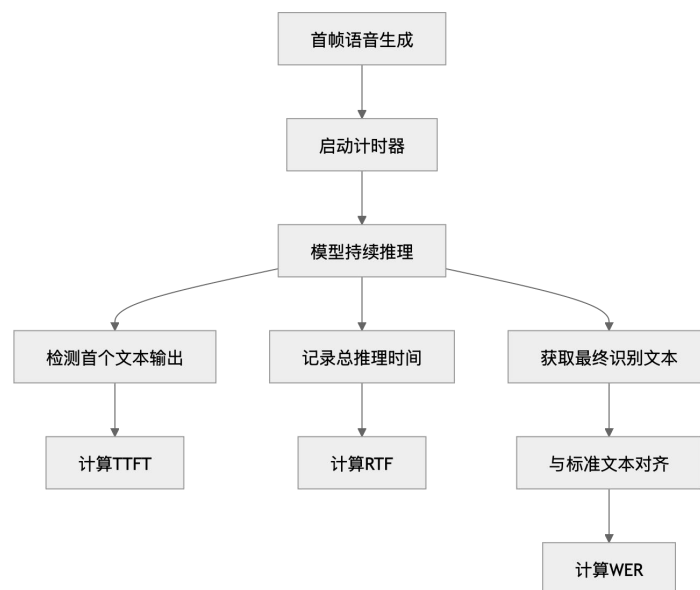


图 5-4 指标计算与统计模块运行流程图

5.5. 结果输出模块设计

结果输出模块负责对各项指标结果进行统一整理与呈现。系统在每个 WAV 文件处理完成后生成对应的单文件指标结果，包括 WER、RTF

与 TTFT 数值，并在批量测试结束后对所有文件的指标进行统计汇总，计算平均值与整体性能数据。输出形式包括控制台日志输出以及结构化数据记录方式，便于后续绘图分析或报告生成。

关键类定义：ResultExporter：

```
1 class ResultExporter:
2
3     function export_console(summary):
4         print(summary)
5
6     function export_file(summary, filepath):
7         write_to_file(summary, filepath)
8
9     function format_report(metrics):
10         return formatted_text
```

该模块在系统架构中位于最底层，承接指标计算模块产生的数据，并将其转化为可读、可分析的结果形式。通过统一的结果管理机制，系统实现了从评测调度到性能评估再到数据输出的完整闭环，运行流程如图 5-5 所示。

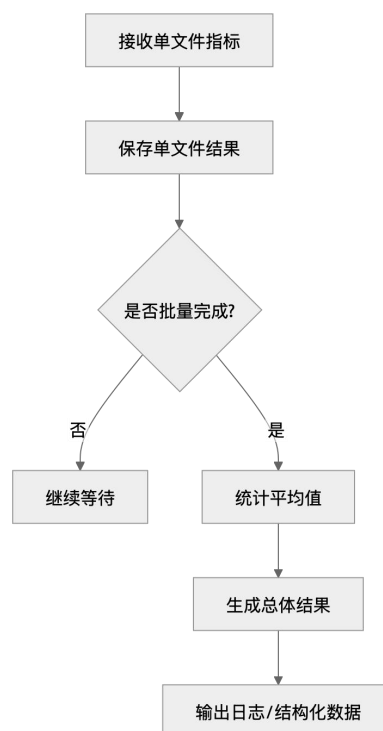


图 5-5 结果输出模块运行流程图

6. 系统关键技术实现

本系统围绕实时会议语音转写场景中“低延迟、高准确、可并发”的核心目标，在整体架构设计基础上，重点实现了流式高保真仿真机制、准实时分片调度机制、滑动窗口优化算法、多粒度流式解码机制以及并行实时指标采集机制等关键技术。各项技术协同作用，使系统能够在离线环境下精确复现真实会议字幕场景，并实现多模型、多粒度、多并发条件下的统一评测。

6.1. Faster-Whisper 准实时分片执行机制

Faster-Whisper 本质属于离线自动语音识别模型，为适配实时字幕场景，本系统通过分片连续调用策略构建准实时执行机制。系统将音频按 1 秒为单位进行切片，并对每个分片独立调用模型进行推理。识别结果在每个分片完成后立即输出，从而实现阶段性实时文本更新效果，如图 6-1 所示。

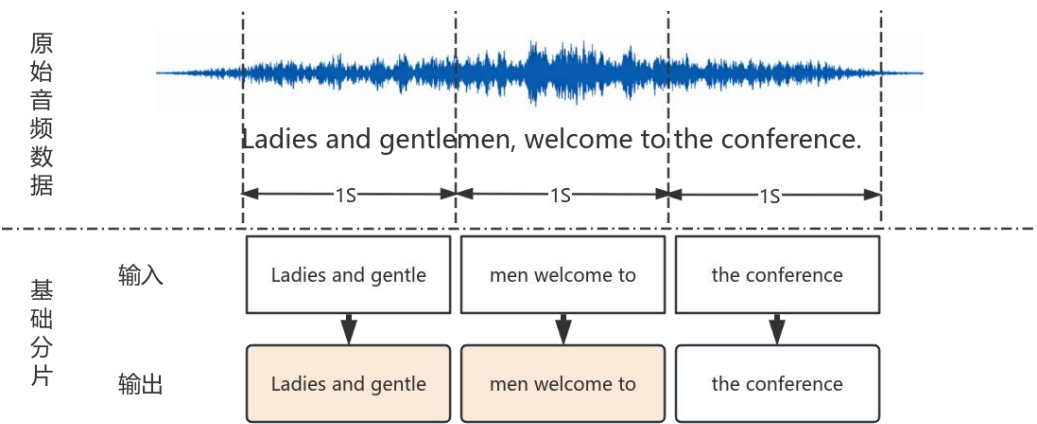


图 6-1 Faster-Whisper 准实时分片执行机制示意图

为保证上下文连续性，系统在分片处理过程中保留历史文本结果并进行拼接处理，使输出文本在时间维度上保持递进关系。该机制使离线模型在无需改动内部结构的情况下实现近似实时应用效果，降低系统改造成本。

该策略的优势在于实现简单、计算资源利用率稳定，但由于分片边界可能截断语义结构，因此在复杂语句场景下可能产生识别误差。

6.2. 滑动窗口重叠优化算法

为缓解 1 秒分片准实时模式下可能出现的语句边界截断问题，本系统设计并实现了基于时间戳动态裁剪的滑动窗口重叠优化算法，如图 6-2 所示。

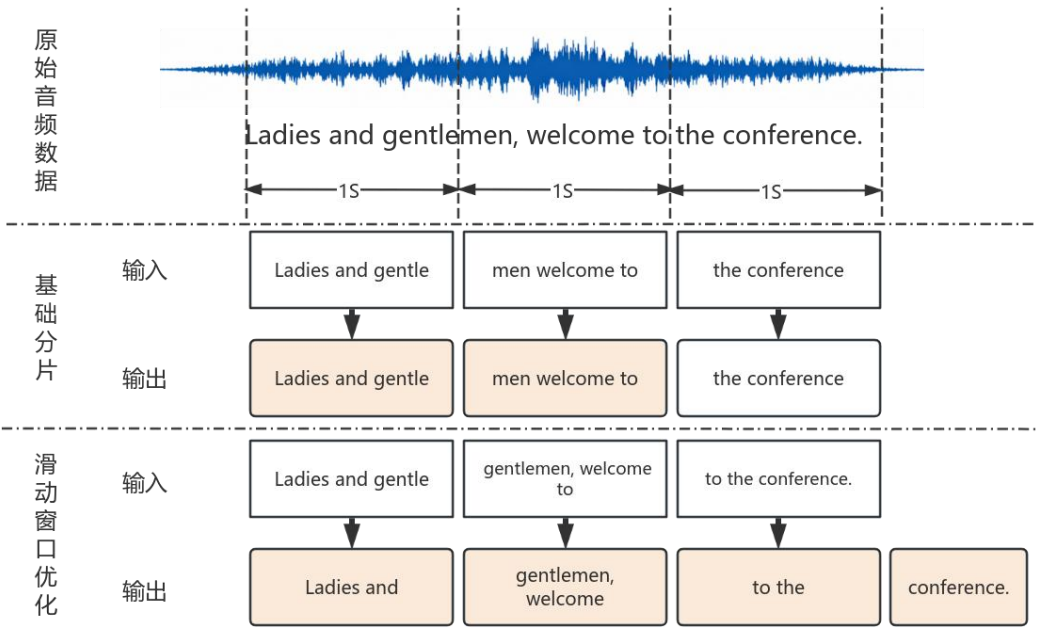


图 6-2 滑动窗口优化算法示意图

在常规分片机制下，每个分片独立推理，分片边界可能将未完成语句截断，影响识别连续性。为此，本系统在分片之间引入可变重叠

机制：在完成当前分片推理后，并非直接清空音频缓冲区，而是基于字级时间戳计算已确认完成的语音区间，仅裁剪已稳定输出部分，保留末尾若干词对应的音频片段作为上下文输入下一轮推理。具体策略包括：

- a) 默认保留最后 1 个识别词作为重叠区；
- b) 若尾词为未完成符号（如“-”），自动增加保留长度；
- c) 在达到强制刷新阈值或文件结束时提交全部内容；
- d) 截断时间基于识别结果中的词级时间戳动态计算。

该算法本质上是一种“基于时间对齐的动态滑动窗口裁剪机制”，在不改变 1 秒分片节奏的前提下增强上下文连续性，提高长语句识别稳定性。由于保留部分历史音频进行重复计算，系统亦能够评估该优化策略对实时性能与并发能力的影响。

6.3. Sherpa-ONNX+BERT 语义增强机制

在 Sherpa-ONNX 流式解码基础上，本系统实现了流式语义增强机制。该模式在获得阶段性识别文本后，引入 BERT 模型对文本进行语义修正与结构优化。语义增强过程与流式解码相衔接，在不显著增加延迟的前提下改善文本可读性与语法完整性，如图 6-3 所示。

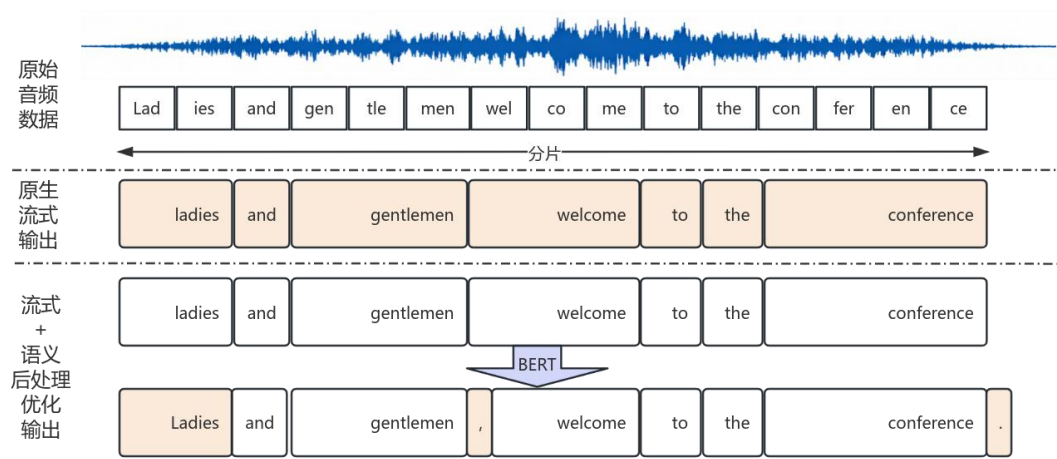


图 6-3 基于 BERT 的语义后处理优化示意图

系统在实现过程中采用异步处理方式，使 BERT 优化过程与主流式解码流程相对独立，从而减少对首字上屏延迟的影响。通过将 Sherpa-ONNX+BERT 作为独立执行模式，系统能够评估语义后处理策略对准确率与实时性能的综合影响。

该机制为工业级会议字幕系统提供了语义层面的增强路径，也为模型对比研究提供了扩展维度。

6.4. 并行实时指标采集与统计机制

为保证不同模型执行模式之间的指标可比性，本系统设计了统一的实时指标采集机制。指标模块采用并行监听策略，自首个音频分片生成时即启动计时器，持续监听模型输出事件并记录关键时间节点。如图 6-4 所示。

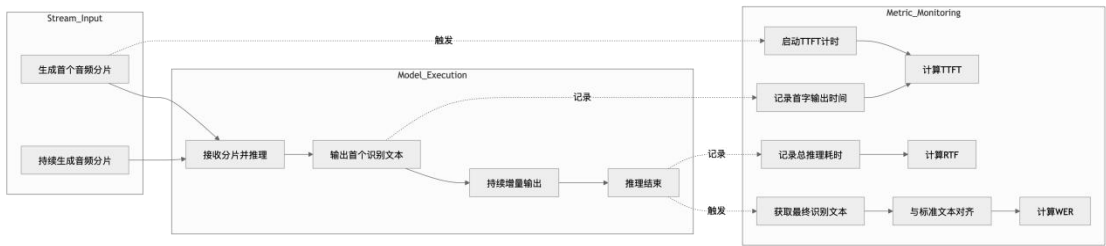


图 6-4 指标采集与统计流程图

TTFT 的统计起点统一定义为首个音频分片生成时刻，终点为首个有效文本输出时刻；RTF 根据模型总推理耗时与音频总时长计算比值；WER 通过对识别结果与标准文本进行对齐计算插入、删除与替换错误数量获得。

该机制贯穿整个模型执行过程，而非事后统计，从而保证时间类指标的精确性与一致性。无论采用 1 秒分片模式还是 0.1 秒流式模式，系统均使用统一时间口径进行统计，确保横向对比的公平性。

7. 系统运行流程

本系统采用统一调度、分层执行、并行监听的运行机制。在实际运行过程中，各功能模块按照既定顺序协同工作，形成从评测配置到结果输出的完整闭环流程。系统运行流程既支持单流延迟测试场景，也支持多流并发压力测试场景，整体执行逻辑保持一致，仅在并发数量与分片粒度参数上存在差异。

系统运行开始于评测调度控制模块。首先由调度模块读取评测配置参数，包括模型执行模式选择、数据集选择、评估指标选择以及并发数设置等信息。系统根据所选模型模式自动确定音频分片粒度策略：当选择 Faster-Whisper 系列模式时，分片粒度设置为 1 秒；当选择 Sherpa-ONNX 系列模式时，分片粒度设置为 0.1 秒。随后系统完成模型加载与运行环境初始化，并根据并发数参数创建对应数量的流式任务实例。

在完成初始化后，系统进入流式语音仿真阶段。流式语音仿真模块读取指定数据集中的 WAV 音频文件，并按照设定的分片粒度进行切割处理。分片数据通过时间节流机制按真实时间节奏逐段输出。在并发场景下，系统通过协程或轮询调度机制推动多路音频流交替运行，使各流实例在时间维度上保持相对独立且并行推进。

当音频分片开始生成时，指标计算与统计模块同步启动监听机制。系统在首个音频分片输出时启动 TTFT 计时器，并持续监听模型输出事件。在模型执行模块中，系统根据所选执行模式调用对应算法流程进行推理计算。Faster-Whisper 模式下采用 1 秒分片准实时推理或滑动窗口优化推理；Sherpa-ONNX 模式下采用 0.1 秒流式增量解码；Sherpa-ONNX+BERT 模式在流式解码基础上进行语义增强处理。模型在推理过程中持续输出识别文本结果，指标模块同步记录关键时间节点。

当模型首次输出有效识别文本时，系统记录首字输出时间并计算 TTFT 指标；当音频处理完成且推理结束时，系统记录总推理耗时并计算 RTF 指标；在最终识别文本生成后，系统将识别结果与标准文本进行对齐计算，从而得到 WER 指标。上述指标统计过程与模型执行过程并行进行，不会阻塞主流程。

在所有音频文件处理完成后，系统进入结果汇总阶段。结果输出模块对每个音频文件的单项指标进行整理记录，并在批量测试结束后自动计算整体平均指标与综合统计数据。最终结果以日志输出或结构化数据形式呈现，供后续分析与对比研究使用。

以 Sherpa-ONNX 模型，LibriSpeech 数据集，评测 TTFT 指标，49 并发为例的输出结果为例：

```
1 -----
2 Sherpa-onnx 并行 TTFT 测试
3 并发数 (Threads): 49
4 计算公式: TTFT = (Chunks * 0.1s) + Parse_Time
5 模型加载完成。
6 -----
7 正在扫描文件列表: /opt/Audio_Datasets/LibriSpeech_WAV/test-clean ...
8 共找到 2620 个音频文件。开始并行处理...
9 文件名 | 分片数 | 计算耗时(s) | TTFT(s)
10 -----
11 7021-85628-0020.wav | 11 | 0.2092 | 1.3092
12 7021-79740-0009.wav | 8 | 0.7051 | 1.5051
13 7021-85628-0021.wav | 11 | 0.2298 | 1.3298
14 7021-79740-0014.wav | 8 | 0.5108 | 1.3108
15 7021-79740-0008.wav | 11 | 0.2694 | 1.3694
16 7021-79759-0003.wav | 14 | 0.3691 | 1.7691
17 .....
18 3575-170457-0030.wav | 11 | 0.0368 | 1.1368
19 -----
20 TTFT 测试完成
21 并发线程数: 49
22 成功处理文件数: 2620
23 评测总耗时 (Wall Time): 48.38 秒
24 =====
25 平均首字延迟 (Average TTFT): 1.3407 秒
26 =====
```

整个运行流程形成“参数配置—环境初始化—流式仿真—模型执行—实时监听—指标计算—结果输出”的闭环结构，如图 7-1 所示。该运行机制能够在不同模型模式与不同并发负载条件下保持一致的统计口径与执行逻辑，从而保证评测结果的稳定性与可比性。

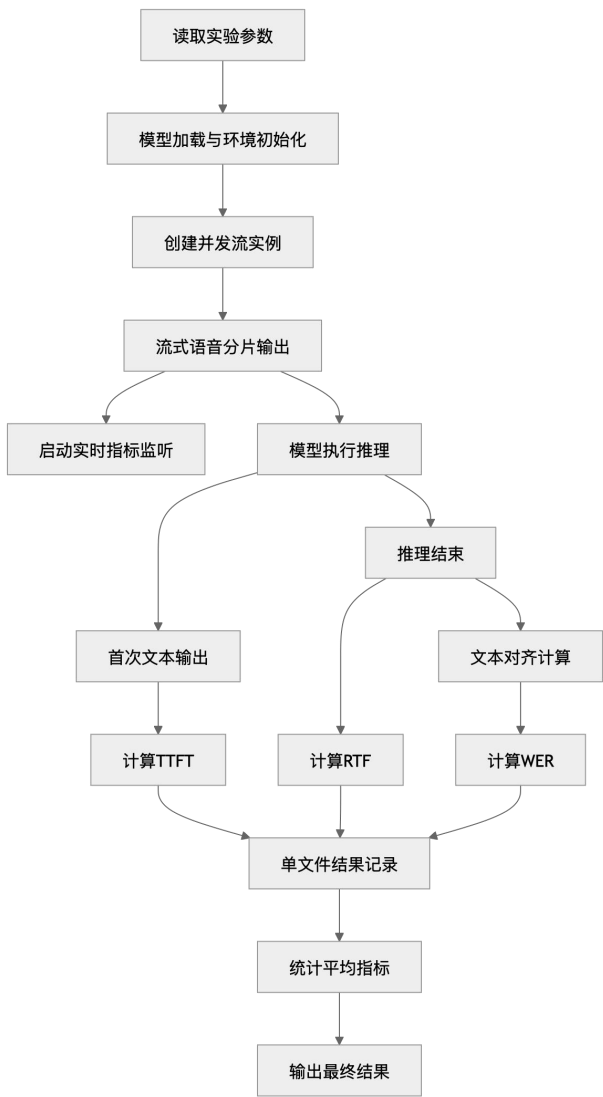


图 7-1 系统运行流程图

8. 软件特点与创新点

8.1. 离线流式仿真技术

本系统提出基于音频分片与时间节流控制的离线流式仿真方法，在无需真实麦克风或网络传输环境的条件下，复现连续语音流输入特征。通过精确控制分片粒度与帧间时间间隔，实现高可重复性的实时性能测试环境。

8.2. 跨架构统一评测框架

系统构建兼容流式 Transducer 架构与分片式 AED 架构的统一评测底座，将不同模型执行模式抽象为统一接口，在相同硬件与延迟约束条件下实现公平对比，保证不同技术路线之间的可比性。

8.3. 语义后处理性能损耗量化机制

区别于仅关注识别精度的传统工具，本系统将语义后处理模块作为独立执行模式，支持动态开启或关闭 BERT 语义增强功能，并实时统计其对 TTFT 与 RTF 的影响，从而量化语义优化所带来的计算开销。

8.4. 高并发负载仿真能力

系统采用基于协程调度的多流轮询机制，在单进程内构建逻辑多实例执行环境，模拟 GPU 高负载运行状态。通过动态提升并发数量并实时采集关键指标，实现模型实时并发能力的工程级评估。

9. 结语

本文档系统阐述了软件的设计思路、结构组成及关键技术实现，完整地反映了软件的开发过程。